Foundations III: Exceptions and Namespaces

Part A: Short Answer Questions

1. Explain the differences between syntax, runtime, and logic errors.

2. State four limitations of traditional error handling techniques.

3. Describe the purpose of try, throw, catch, and handle.

4. Differentiate between std::exception, std::logic_error, std::invalid_argument, std::out_of_range, and std::runtime_error.

5. What is stack unwinding?

6. Define scope and list four variable scopes discussed in the slides.

7. How do namespaces solve naming conflicts?

8. Differentiate between named and unnamed namespaces.

9. What is the purpose of a namespace alias?

10. Explain the use of constexpr in namespaces.

Part B: Long Answer Questions

1. Discuss traditional error handling methods and explain why exceptions are preferred in Modern C++.

2. Using suitable examples, explain the C++ exception hierarchy and when each exception type should be used.

3. Describe the process of stack unwinding and explain its relationship with RAII.

4. Explain how namespaces support large-scale software development and library organization.

5. Compare named namespaces, unnamed namespaces, nested namespaces, and namespace aliases with examples.

Foundations IV and V : Defensive Programming and Time

Part A: Short Answer Questions

1. Define defensive programming.

2. List four differences between older C-style and Modern C++ defensive programming.

3. When should assert() be used?

4. When should assert() NOT be used?

5. What is a function precondition?

6. Compare std::time and std::clock.

7. Why is std::chrono preferred in Modern C++?

8. Differentiate between wall-clock time and CPU time.

9. State the purpose of std::this_thread::sleep_for().

10. Give two real-world uses of sleep_for().

Part B: Long Answer Questions

1. Compare older C-style defensive programming with Modern C++ defensive programming.

2. Explain the role of assertions in defensive programming and provide examples of appropriate and inappropriate use cases.

3. Compare std::time, std::clock, and std::chrono in terms of precision, purpose, and modern relevance.

4. Describe how <thread> and <chrono> work together in Modern C++ applications.

5. Develop a discussion on how defensive programming techniques improve software reliability and maintainability.

Foundations IV and V: Pre-processing, Macros, Makefiles, Debuggers and Profilers

Part A: Short Answer Questions

1. What is a preprocessor directive?

2. State the purpose of #include.

3. What is conditional compilation?

4. What are header guards used for?

5. Compare #pragma once and traditional header guards.

6. Define an object-like macro and a function-like macro.

7. Why should macros be used sparingly in Modern C++?

8. State four purposes of a Makefile.

9. Differentiate between .cpp, .o, and executable files.

10. What is the role of a debugger and a profiler in software development?

Part B: Long Answer Questions

1. Explain the role of preprocessing in C++ and discuss the importance of conditional compilation.

2. Discuss the advantages and disadvantages of macros. Include Modern C++ alternatives.

3. Explain the complete compilation and build process from source files to executable program.

4. Using a suitable example, explain the structure and components of a Makefile.

5. Compare the roles of debuggers and profilers in improving software quality and performance.